

Atomic floating-point min/max

Document number: P3008R1.

Date: 2024-01-29.

Authors: Gonzalo Brito Gadeschi, David Sankel <dsankel@adobe.com>.

Reply to: gonzalob at nvidia.com .

Audience: LEWG, SG6.

Table of Contents

- [Atomic floating-point min/max](#)
- [Changelog](#)
- [Introduction](#)
- [Survey of programming language floating-point min/max API semantics](#)
 - [C++ std::min/std::max](#)
 - [C fmin/fmax](#)
 - [C23 minimum/maximum/minimumNumber/maximumNumber](#)
- [Impact of replacing min with fminimum_num](#)
- [Survey of hardware atomic floating-point min/max API semantics](#)
 - [Performance impact of atomic<floating-point>::fetch_min/_max semantics](#)
- [Design space](#)
 - [Alternatives that reserve fetch_min/fetch_max for std::min/std::max](#)
 - [Alternatives that provide fetch_min/fetch_max with different semantics](#)
- [Unresolved questions](#)
- [Wording](#)
- [References](#)

Changelog

- **Revision 1:**
 - Update to account for removal of min/max from P0493.
 - Update C17 to C23 wording for signaling NaNs.
 - SG1 DIRECTION POLL:
 1. The default `fetch_min(T, mo=SC)->T`, `fetch_max(T, mo=SC)->T` should exist

2. The default `fetch_min`, `fetch_max` should have these semantics:
 - * Which value is stored in the atomic is unspecified if an input is a NaN
 - * Implementations should treat -0 as less than +0 (use normative encouragement)
 - * (Note: must *not* require `std::min/max`)
3. Users need a way to specify semantics (up to LEWG how to achieve this):
 - * Must-have: `fminimum/fmaximum`
 - * Must-have: `fminimum_num/fmaximum_num`
 - * Can have if LEWG desires it but we do not: `std::min/max`

SF	F	N	A	SA
5	6	2	0	0

Unanimous consent

- **Revision 0:** initial revision.

Introduction

[P0493R4](#) *Atomic minimum/maximum* proposes the addition of `atomic<T>::fetch_min` and `atomic<T>::fetch_max` operations, which atomically perform `std::min` and `std::max` computations.

The Varna '23 plenary rejected these semantics for `atomic<floating-point>` types due to safety concerns. These semantics deviate from the current standard practice, as defined by [IEEE 754-2019](#) recommendations and the capabilities of available hardware.

These operations were removed from subsequent revisions of [P0493](#). This paper proposes adding these operations to C++ with different semantics. It reviews the semantics proposed in [P0493R4](#) and standard practice: [IEEE 754-2008](#), [IEEE 754-2019](#), and available hardware support. It then explores the design space and evaluates alternative solutions.

Finally, the authors propose two coherent alternatives and concrete wording changes implementing some of the semantics [IEEE 754-2019](#) recommends all vendors implement.

Survey of programming language floating-point min/max API semantics

This section provides an overview of widely used floating-point `min / max` semantics. We will focus on how various `min / max` semantics handle the following "corner cases":

- **Signed zero:** Should `-0` be considered less than `+0` (`-0 < +0`), or should `-0` and `+0` be treated as equivalent (`-0 == +0`) such that, e.g., `min(+0, -0) = +0` ?
- **One quiet NaN (qNaN):** when one of the arguments is a qNaN, should it be treated as *Missing Data* and the other argument returned, or should it be treated as an *Error* and propagated?
 - **Missing Data:** returns the other argument: `min(x, qNaN) = x` and `min(qNaN, x) = x` .
 - **Error:** propagates the qNaN: `min(x, qNaN) = qNaN` and `min(qNaN, x) = qNaN` .

Explicitly and intentionally, this paper ignores:

- **Signaling NaNs:** C23 specifies that their full support is not required, and where their specification is not provided leaving them as implementation-defined behavior (either as quiet NaN or signaling NaN). C++ does not specify anything about them beyond what C23 covers. An analysis of signaling NaNs and their impact on this proposal is left to a future version of this paper and captured as a current unresolved question in a latter section of this proposal.
- **NaN payload propagation:** IEEE 754 does not mandate it, and programming languages do not interpret NaN payloads.

Table 1 summarizes the semantics of the different C or C++ APIs regarding Signed Zeros and Quiet NaN handling and documents whether implementations that implement particular IEEE 754 semantics are valid implementations of the APIs.

C API	C++ API	Signed Zeros	One qNaN	IEEE 754 impl valid?
-	<code>min</code> <code>max</code>	Equivalent	Undefined [1]	Ternary [0]
<code>fmin</code> <code>fmax</code>	-	Equivalent or <code>-0 < +0</code> (Qol) [2]	Missing Data	<code>minNum</code> <code>maxNum</code> or (Qol) <code>minimumNumber</code> <code>maximumNumber</code>
<code>fminimum</code> <code>fmaximum</code>	-	<code>-0 < +0</code>	Error	<code>minimum</code> <code>maximum</code>
<code>fminimum_num</code> <code>fmaximum_num</code>	-	<code>-0 < +0</code>	Missing Data	<code>minimumNumber</code> <code>maximumNumber</code>

[0] Ternary semantics: `min(x,y) = y < x? y : x`, `max(x,y) = x < y? y : x`; both return the first argument if the arguments are equivalent or one argument is a NaN.

[1] In practice, the same implementation as for valid values is used, and the first argument passed to the function is returned (the second argument of the ternary expression is picked, since the conditional involving a NaN is always false).

[2] Qol: "Quality of Implementation", i.e., `fmin` does not require `-0 < +0`, but it recommends that high quality implementations implement it.

C++ `std::min` / `std::max`

The `std::min` and `std::max` algorithms have a *precondition* on their arguments ([\[alg.min.max.1\]](#)):

Preconditions: For the first form, `T` meets the [Cpp17LessThanComparable](#) requirements (Table [Cpp17LessThanComparable](#)).

which NaNs do not satisfy.

► Rationale.

For valid values, implementations follow ([\[alg.min.max\]](#)):

`[ std::min ]:`

Returns: The smaller value. Returns the first argument when the arguments are equivalent.

`[ std::max ]:`

Returns: The larger value. Returns the first argument when the arguments are equivalent.

which preserves equivalence between `-0` and `+0` ([godbolt](#)):

```
// Listing 1: std::min/std::max
auto min = [](auto& a, auto& b) -> auto& { return std::min(x, y); }; // y < x
auto max = [](auto& a, auto& b) -> auto& { return std::max(x, y); }; // x < y

min(qNaN, 2.f); // UB: qNaN
max(qNaN, 2.f); // UB: qNaN
min(2.f, qNaN); // UB: 2
max(2.f, qNaN); // UB: 2
min(-0.f, +0.f); // -0
max(-0.f, +0.f); // -0
min(+0.f, -0.f); // +0
max(+0.f, -0.f); // +0
```

In all standard library implementations surveyed, the manifestation of the *undefined behavior* is to return the first argument; this is depicted in the example with "UB: qNaN" and "UB: 2".

The behavior of these semantics are:

- **Signed Zero:** equivalent.
- **One qNaN:** undefined.

In concurrent programs, these semantics become even less intuitive, e.g., the sign of the result of a concurrent computation may depend on the order in which an observer sitting on top of the memory location observes the operations from different threads. If negative and positive zero are equivalent, the sign of this final result may differ across executions.

C `fmin` / `fmax`

The semantics of the [ISO/IEC 9899:2024 \(C23\) standard](#) `fmin` / `fmax` functions are compatible with implementations of [IEEE 754-2008](#) `minNum` and `maxNum` and of [IEEE 754-2019](#) `minimumNumber` / `maximumNumber`. The `fmin` / `fmax` functions are available in C++ through the `<cmath>` header as `std::fmin/std::fmax`.

The semantics of C's `fmin` / `fmax`

- **Signed zero:**
 - `-0` may be equivalent to `+0` (`minNum` / `maxNum`)
 - `-0 < +0` allowed as QoI (`minimumNumber` / `maximumNumber` guarantee it).
- **One qNaN:** missing data (other value returned).

- ▶ (collapsible) C23 `fmin`/`fmax` specification.
- ▶ (collapsible) [IEEE 754-2008] `minNum`/`maxNum` specification.

That is, in the presence of a qNaN, these return the value, but signed zeros may or may not be equivalent:

```
// Listing 2: fmin/fmax
fmin(qNaN, 2.f): 2
fmax(qNaN, 2.f): 2
fmin(2.f, qNaN): 2
fmax(2.f, qNaN): 2
fmin(-0.f, +0.f): -0 or +0
fmax(-0.f, +0.f): -0 or +0
fmin(+0.f, -0.f): -0 or +0
fmax(+0.f, -0.f): -0 or +0
```

[IEEE 754-2019](#) removed `minNum` and `maxNum` operations and replaced them with `minimumNumber` / `maximumNumber` and `minimum` / `maximum`. These are surveyed in the next section. The gist of the rationale for their removal from [The Removal/Demotion of `MinNum` and `MaxNum` Operations from IEEE 754-2018](#) is:

These [`minNum`, `maxNum`] operations are removed from or demoted in IEEE std 754-2018, due to their non-associativity. [...] With this non-associativity, different compilations or runs on parallel processing can return different answers [...].

C23

`minimum` / `maximum` / `minimumNumber` / `maximumNumber`

[IEEE 754-2019](#) removed `minNum` / `maxNum` and recommends programming languages to provide their replacements instead: `minimumNumber` / `maximumNumber` / `minimum` / `maximum`.

- ▶ (collapsible) IEEE 754-2019 `minimum`/`maximum`/`minimumNumber`/`maximumNumber` specification.
- ▶ (collapsible) C23 `fmaximum`/`fminimum`/`fmaximum_num`/`fminimum_num` specification.

The semantics of these functions matches for signed zero: `-0 < +0` , and differs in their treatment when one argument is a qNaN:

- **Missing Data:** `fminimumNumber` / `fmaximumNumber` , return the Number.
- **Errors:** `fminimum` / `fmaximum` , propagate the qNaN.

```
// Listing 3: fminimum/fmaximum
fminimum(qNaN, 2.f): qNaN
fmaximum(qNaN, 2.f): qNaN
fminimum(2.f, qNaN): qNaN
fmaximum(2.f, qNaN): qNaN
fminimum(-0.f, +0.f): -0
fmaximum(-0.f, +0.f): +0
fminimum(+0.f, -0.f): -0
fmaximum(+0.f, -0.f): +0

fminimum_num(qNaN, 2.f): 2
fmaximum_num(qNaN, 2.f): 2
fminimum_num(2.f, qNaN): 2
fmaximum_num(2.f, qNaN): 2
fminimum_num(-0.f, +0.f): -0
fmaximum_num(-0.f, +0.f): +0
fminimum_num(+0.f, -0.f): -0
fmaximum_num(+0.f, -0.f): +0
```

Impact of replacing min with fminimum_num

Table 2 captures the impact of replacing `min` with `fminimum_num` is as follows:

std::min / std::max	fminimum_num / fmaximum_num
min(qNaN, 2.f); // UB: qNaN	fminimum_num(qNaN, 2.f); // 2
max(qNaN, 2.f); // UB: qNaN	fmaximum_num(qNaN, 2.f); // 2
min(+0.f, -0.f); // +0	fminimum_num(+0.f, -0.f); // -0
max(-0.f, +0.f); // -0	fmaximum_num(-0.f, +0.f); // +0

That is, when the first input of `std::min` / `std::max` is a qNaN, then these switch from exhibiting *undefined behavior* to returning a number, and when signed zeros are involved, there is a case where the result has a different sign.

Survey of hardware atomic floating-point min/max API semantics

On systems without native support for atomic floating-point min/max operations, these operations must be performed atomically, e.g., using a CAS loop, a compare-and-conditional-store loop, an LL/SC loop, or, e.g., by taking a lock, which would make the atomic not lock-free. In these systems, the memory latency overhead may outweigh the cost of performing the actual *arithmetic* portion of the min/max operations, and extra effort may be required to ensure forward progress properties like starvation freedom, and therefore those systems are not considered here.

Table 3 surveys the publicly known Instruction Set Architectures (ISAs) with atomic floating-point min/max operations, which are all GPU ISAs:

Vendor	ISA	Instructions	IEEE-2019 compat	Signed Zero	Quiet NaN
AMD	CDNA2+	MIN MAX	<code>minimumNumber</code> <code>maximumNumber</code>	$-0 < +0$	Missing Data
Intel	Xe ISA	<code>AOP_FMIN</code> <code>AOP_FMAX</code> [0]	<code>minimumNumber</code> <code>maximumNumber</code>	$-0 < +0$	Missing Data
NVIDIA	PTX	atom red	<code>minimumNumber</code> <code>maximumNumber</code>	$-0 < +0$	Missing Data
Neutral [1]	SPIR-V Extension	OpAtomicFMinEXT OpAtomicFMaxEXT	C <code>fmin / fmax</code>	Equivalent; QoI: $-0 < +0$	Missing Data

- [0]: Volume 2d: Command Reference: Structures, page 229.
- [1]: The Vulkan extension corresponding to this SPIR-V extension is `VK_EXT_shader_atomic_float2`. Hardware that implements it is listed [here](#) and [here](#).

All the architectures surveyed order $-0 < +0$, treat qNaNs as missing data, and implement IEEE 754-2019 `minimumNumber` and `maximumNumber` semantics. The SPIR-V extension requires C `fmin / fmax` semantics, which allows $-0 < +0$ but does not require it.

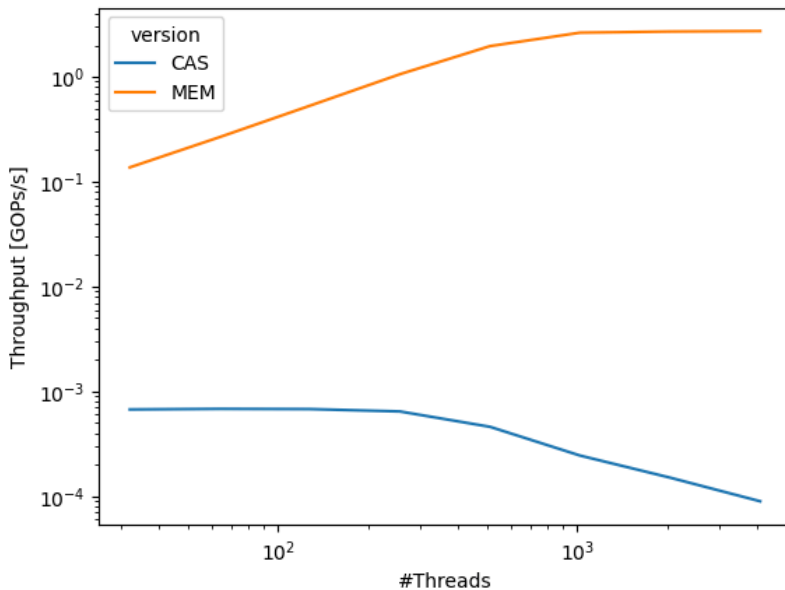
The semantics vendor implement are compatible with C23's `fminimum_num / fmaximum_num`, and C's `fmin / fmax`, but not with C++'s `std::min / std::max` due to the different outcomes when signed-zeros are involved.

Performance impact of `atomic<floating-point>::fetch_min / _max` semantics

We compare the performance impact of two different `atomic<floating-point>::fetch_max` semantics:

- `std::min`, which lowers to a CAS-loop that performs `std::min` (similar for compare-and-conditional-store), and
- `fminimum_num`, which lowers to native atomic operations,

using a synthetic micro-benchmark that measures throughput in Giga Operations per Second (y-axis; logarithmic) as a function of the number of threads (x-axis; logarithmic) from 32 to 130'000 hardware threads on an NVIDIA GPU system.



Modern concurrent systems provide dozens of millions of hardware threads operating on the same shared memory.

While native in-memory atomic operations increase throughput until its theoretical peak with just a few thousand threads, the performance of compare and swap strategies decreases as the number of threads increases due to excess contention. At just a few thousand threads, the performance is already four orders of magnitude worse (~10'000x) than that of native in-memory operations. This is why vendors of highly concurrent systems provide these operations.

While most CPU ISAs lack native hardware instructions for atomic floating-point min/max, their performance impact is expected to be similar to that of atomic integer min/max. Section 9 of [P0493R4](#) presents benchmarks comparing the performance of a CAS-based implementation against an Arm v8.1 implementation using the native atomic `ldsmxl` instruction. The benchmark covers a range of cores, from 2 to 64. The performance improvement of the native

instruction ranges between 2.75x (2 cores) to 1.7x at 64 cores. We expect this behavior to transfer to atomic floating-point min/max.

We, unfortunately, do not have benchmarks for other hardware configurations at this time.

Design space

The design space can be categoratized into two groups of alternatives:

- **Same semantics:** `::fetch_min / _max` have *same* semantics as `std::min / std::max`,
- **Different semantics:** `::fetch_min / _max` have *different* semantics than `std::min / std::max`.

The following table shows the subjectively most reasonable alternative of each category side-by-side, along with a summary of their advantages and disadvantages, which are analyzed in the following section.

Table 4: Comparison of best alternative of each category.

Category	<code>std::min</code> semantics	<code>std::fminimum_num</code> semantics
Concrete Example	<code>x.fetch_min(y);</code> <code>x.fetch_fminimum_num(y);</code>	<code>x.fetch_min(y,</code> <code>std::less{});</code> <code>x.fetch_min(y);</code>
Pros	- Parallelizing preserves semantics: NaNs are UB & <code>-0 == +0</code> . - Consistency in standard.	- Safer semantics by default. - Hardware acceleration by default.
Cons	- The convenient name, <code>min</code> , has unportable behavior. - The convenient name has degraded performance.	- Inconsistency in identically named functions. - Subtle surprises when transitioning to atomics.
Training reqs.	Misuse and Performance.	Subtle surprising behavior change for floating-point.
Default	Stability.	Correctness + Performance.
Opt-in	Correctness + Performance.	Stability.

C++ needs to make an engineering trade-off: prioritizing consistency in the standard or prioritizing better defaults.

Alternatives that reserve `fetch_min / fetch_max` for `std::min / std::max`

In this group of alternatives, exposure of different semantics shall happen through other APIs, and we may potentially expose one or more of the following:

- `fetch_min / fetch_max` with `std::min / std::max` semantics, or
- `fetch_fminimum / fetch_fmaximum` with `fminimum / fmaximum` semantics, or
- `fetch_fminimum_num / fetch_fmaximum_num` with `fminimum_num / fmaximum_num` semantics, or
- `fetch_min / fetch_max` API accepting a defaulted comparison function object `Cmp` that satisfies the requirements of [Compare](#) and defaults to `std::less`, potentially providing other function objects to choose `minimum / minimumNumber` semantics.

```

// Listing 4: Same semantics as std::min/std::max.
// One or more of the below options

template <floating-point> struct less_fminimum;
template <floating-point> struct less_fminimum_num;
template <floating-point> struct less_fmaximum;
template <floating-point> struct less_fmaximum_num;

template <floating-point>
class atomic {
    using T = floating-point;

    T fetch_min(T x, memory_order = memory_order_seq_cst); // std::min
    T fetch_max(T x, memory_order = memory_order_seq_cst); // std::max

    T fetch_fminimum(T x, memory_order = memory_order_seq_cst); // std::fmin
    T fetch_fmaximum(T x, memory_order = memory_order_seq_cst); // std::fmax

    T fetch_fminimum_num(T x, memory_order = memory_order_seq_cst); // std::fmin
    T fetch_fmaximum_num(T x, memory_order = memory_order_seq_cst); // std::fmax

    template <typename Compare>
    T fetch_max(T other, Compare cmp = std::less, memory_order ord = memory_order_seq_cst);
    template <typename Compare>
    T fetch_min(T other, Compare cmp = std::less, memory_order ord = memory_order_seq_cst);
};

x.fetch_min(y); // std::min
x.fetch_min(y, less_fminimum{}); // std::fminimum

```

The advantage of these alternatives is that they preserve the semantics of sequential code that uses `std::min` when it is made concurrent via `fetch_min`.

The disadvantages of these alternatives are that the `fetch_min / _max` semantics are unintuitive for floating-point numbers and error prone, and do not benefit from any hardware acceleration available as surveyed in **Table 2**.

Generic programmers willing to avoid these disadvantages need to opt-in to wrapping these APIs as follows:

```
// Listing 5: atomic min wrapper
template <typename T>
T atomic_min_wrapper(atomic<T>& x, T& y) {
    if constexpr(std::is_floating_point_v<T>) {
        return x.fetch_fminimum(y);
    }
    return x.fetch_min(y);
}
```

But creating such wrappers requires developers to be aware of essentially everything discussed in this proposal until this point.

Such awareness may be enforced by not providing `fetch_min` / `fetch_max` APIs for floating-point atomics at all, and instead only provide the `fetch_fminimum` and related APIs, requiring developers to explicitly pick the semantics, and making wrappers like the one in **Listing 5** mandatory for writing generic code.

Alternatives that provide `fetch_min` / `fetch_max` with different semantics

There is precedent in the C++ standard for the atomic operations semantics to deviate from the non-atomic semantics. For example, `atomic<int>::fetch_add` wraps around on overflow, instead of exhibiting undefined behavior, see [\[atomics#ref.int-6\]](#):

Remarks: For signed integer types, the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.

[Note 2: There are no undefined results arising from the computation. — end note]

In a similar spirit, `atomic<floating-point>::fetch_min` / `::fetch_max` operations could be specified to be well-defined for NaNs and to respect `-0 < +0`.

If we restricts the specification to the semantics available in the scalar floating-point functions, we have 3 different options:

- `fminimum` / `fmaximum`: NaNs are propagated as errors and `-0 < +0`.
- NaNs are treated as missing data, i.e., the number is returned when one argument is a NaN, and
 - `fminimum_num` / `fmaximum_num`: `-0 < +0`.

- `fmin / fmax` : `-0 == +0` or (QoI) `-0 < +0` .

The semantics chosen could be complemented with a `fetch_min / fetch_max` API that accepts a comparison object that needs to satisfy the requirements of [Compare](#) but also allows accepting, e.g., one of a set of floating-point-specific blessed comparison-like objects provided by the standard that handle NaNs in specific ways but do not provide a strict weak ordering when NaNs are present. This enables applications to pick whether, e.g., they want to treat NaNs as missing data, or as errors, or whether they want to treat -0 as equivalent to +0, or as +0 greater than -0.

For example, C++ could do some or all of:

```
// Listing 6: Different semantics as std::min/max

template <floating-point> struct less_fminimum;
template <floating-point> struct less_fminimum_num;
template <floating-point> struct less_fmaximum;
template <floating-point> struct less_fmaximum_num;

template <floating-point>
class atomic {
    using T = floating-point;

    T fetch_min(T x, memory_order = memory_order_seq_cst); // std::fminimum or
    T fetch_max(T x, memory_order = memory_order_seq_cst); // std::maximum or

    template <typename Compare>
    T fetch_max(T other, Compare cmp = fminimum_num{}, memory_order ord = memc
    template <typename Compare>
    T fetch_min(T other, Compare cmp = fminimum_num{}, memory_order ord = memc
};

x.fetch_min(y); // std::fminimum_num
x.fetch_min(y, less_fminimum{}); // std::fminimum
x.fetch_min(y, less_fminimum_num{}); // std::fminimum
x.fetch_min(y, less{}); // std::less
```

These groups of alternatives give applications the capability to write generic code which may encounter floating-point values and then exhibits reasonable behavior and benefits from hardware acceleration, without requiring developers of being aware of most of what has been discussed in this paper.

If a developer encounters different results when porting their sequential code using `std::min` to `fetch_min` , discovering those differences is required to notice that the developer may want to

opt-in to the `std::min` semantics. This feedback loop is - for better or worse - more direct than requiring the developer to be a floating-point expert. If the differences are due to NaNs, then the original sequential program already exhibited undefined behavior, but the differences may be due to the treatment of `-0 == +0` in the sequential implementation, which may lead to a change of sign in the final result. This is likely to be a defect in the sequential program, but is well-defined in C++. It is still most-likely to be a defect in the concurrent program.

Unresolved questions

- **Signaling NaNs:** the first revision of this proposal left out signaling NaNs. A future revision of this proposal should include analysis for these and for hardware support.

Wording

Add the following to [\[atomics.ref.float\]](#) immediately below `fetch_sub` :

```
namespace std {
template <> struct atomic_ref<floating-point> {
    floating-point fetch_max(floating-point, memory_order = memory_order::seq_cst) co
    floating-point fetch_min(floating-point, memory_order = memory_order::seq_cst) co
    floating-point fetch_fmaximum(floating-point, memory_order = memory_order::seq_cs
    floating-point fetch_fminimum(floating-point, memory_order = memory_order::seq_cs
    floating-point fetch_fmaximum_num(floating-point, memory_order = memory_order::se
    floating-point fetch_fminimum_num(floating-point, memory_order = memory_order::se
};
}
```

```
floating-point-type fetch_key(floating-point-type operand,
                             memory_order order = memory_order::seq_cst) const noexcept
```

- **Effects:** Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations ([intro.races]).
- **Returns:** Atomically, the value referenced by `*ptr` immediately before the effects.
- **Remarks:** If the result is not a representable value for its type ([expr.pre]), the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on `floating-point-type` should conform to the `std::numeric_limits<floating-`

point-type> traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on floating-point-type may be different than the calling thread's floating-point environment.

- Remarks: For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments. If the given operand or the value referenced by `*ptr` are NaN, which of these values is stored at `*ptr` is unspecified. If the given operand and the value referenced by `*ptr` are differently signed zeros, which of these values is stored at `*ptr` is unspecified.
- Recommended practice: The implementation of `fetch_max` and `fetch_min` should treat `-0` as smaller than `+0`.
- Remarks: For `fetch_fmaximum` and `fetch_fminimum`, the maximum and minimum computation is performed as if by `std::fmaximum` and `std::fminimum`, with the object value and the first parameter as the arguments.
- Remarks: For `fetch_fmaximum_num` and `fetch_fminimum_num`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments.

Add the following to [\[atomics.types.float\]](#) immediately below `fetch_sub` :

```
namespace std {  
template <> struct atomic<floating-point> {  
    floating-point fetch_max(floating-point, memory_order = memory_order::seq_cst) volatile;  
    floating-point fetch_max(floating-point, memory_order = memory_order::seq_cst) nothrow;  
    floating-point fetch_min(floating-point, memory_order = memory_order::seq_cst) volatile;  
    floating-point fetch_min(floating-point, memory_order = memory_order::seq_cst) nothrow;  
    floating-point fetch_fmaximum(floating-point, memory_order = memory_order::seq_cs);  
    floating-point fetch_fmaximum(floating-point, memory_order = memory_order::seq_cs) nothrow;  
    floating-point fetch_fminimum(floating-point, memory_order = memory_order::seq_cs);  
    floating-point fetch_fminimum(floating-point, memory_order = memory_order::seq_cs) nothrow;  
    floating-point fetch_fmaximum_num(floating-point, memory_order = memory_order::seq_cs);  
    floating-point fetch_fmaximum_num(floating-point, memory_order = memory_order::seq_cs) nothrow;  
    floating-point fetch_fminimum_num(floating-point, memory_order = memory_order::seq_cs);  
    floating-point fetch_fminimum_num(floating-point, memory_order = memory_order::seq_cs) nothrow;  
};  
}
```



```
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile noexcept  
T fetch_key(T operand, memory_order order = memory_order::seq_cst) noexcept;
```

- Constraints: For the volatile overload of this function, `is_always_lock_free` is true.
- Effects: Atomically replaces the value pointed to by this with the result of the computation applied to the value pointed to by this and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([intro.multithread]).
- Returns: Atomically, the value pointed to by this immediately before the effects.
- Remarks: If the result is not a representable value for its type ([expr.pre]) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on floating-point-type should conform to the `std::numeric_limits<floating-point-type>` traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on floating-point-type may be different than the calling thread's floating-point environment.
- Remarks: For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments. If the given operand or the object value are NaN, which of these values replaces the object value is unspecified. If the given operand and the object value are differently signed zeros, which of these values replaces the object value is unspecified.
- Recommended practice: The implementation of `fetch_max` and `fetch_min` should treat `-0` as smaller than `+0`.
- Remarks: For `fetch_fmaximum` and `fetch_fminimum`, the maximum and minimum computation is performed as if by `std::fmaximum` and `std::fminimum`, with the object value and the first parameter as the arguments.
- Remarks: For `fetch_fmaximum_num` and `fetch_fminimum_num`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments.

This requires updating the C standard to C23 (do you see the issue?), for which we'd need to modify [\[intro.scope#2\]](#):

C++ is a general purpose programming language based on the C programming language as described in ISO/IEC 9899:20182024 Programming languages — C (hereinafter referred to as the C standard).

and [\[intro.refs\]](#):

(1.3) ISO/IEC 9899:20182024, Programming languages — C

[...]

(1.10.2) The library described in ISO/IEC 9899:20182024, Clause 7, is hereinafter called the C standard library.

so that then we could then add these to [\[cmath.syn\]](#):

```
namespace std {  
constexpr floating-point-type fmaximum num(floating-point-type x, floating-point-ty  
constexpr float fmaximum numf(float x, float y);  
constexpr long double fmaximum numl(long double x, long double y);  
  
constexpr floating-point-type fminimum num(floating-point-type x, floating-point-ty  
constexpr float fminimum numf(float x, float y);  
constexpr long double fminimum numl(long double x, long double y);  
}
```

These changes become possible once the C23 standard is published.

References

1. [P0493R4 Atomic maximum/minimum](#).
2. [The Removal/Demotion of MinNum and MaxNum Operations from IEEE 754™-2018](#).
3. [Min-max functions](#) Proposal for TS 18661 update WG14 N2273.
4. [ISO/IEC 9899:2024 C23 standard](#).
5. [ANSI/IEEE Std 754-2008](#).
6. [ANSI/IEEE Std 754-2019](#).